

Daffodil International University

STUDENT MANAGEMENT SYSTEM PROJECT REPORT

Course Code: ICE 1248

Course Name: Object Oriented Programming Lab

Students Name:

Fahim Shahriar (241-50-002)

Ashiqur Rahman (241-50-009)

Orman (241-50-035)

Section: A

Lecturer's Name: Istakiak Adnan Palash

Date of Submission: 12-12-2024

Table of Contents

- Introduction
- Objectives
- Features
- Requirement Analysis
 - Flow Chart
 - Algorithm
 - Classes
 - Technologies Used
- System Architecture
 - Results
 - Output
- Conclusion
- Future Works

Introduction:

Our goal is to make a program which will help to organize all student's information securely and easily. Many institutions still store all their student's information in register or large excel sheets. This is the conventional way to store student's information. But we wanted to make it even easier and user friendly. Our program can easily store many student's data and can easily search and show a particular student's information within seconds. This makes storing large amounts of data effortless.

Objectives:

- 1) Offer a single platform for efficient management of student records, encompassing contact, academic, and personal data.
- 2) Provide features that allow students' data to be created, viewed, updated, and deleted (CRUD).
- 3) Create an easy-to-use interface that allows users or administrators to communicate with the system.
- 4) Put strong validation procedures in place to guarantee accurate data submission.
- 5) Make it possible to quickly retrieve student data by utilizing search and filter features.
- 6) To manage exceptions during operations, include appropriate error detection and clear warnings.

Features:

Our program is secured as it requires username and password to access data. This program is efficient and user-friendly and requires less time to search and show output.

Moreover this program is easily readable and maintainable and it can store unlimited employee data. Some key features are given below:

Add Student: Allows users to add a new student with details such as ID, name, contact number, course, and department.

View Students: Displays a list of all students currently stored in the system.

Update Student: Enables updating the details of a specific student using their ID.

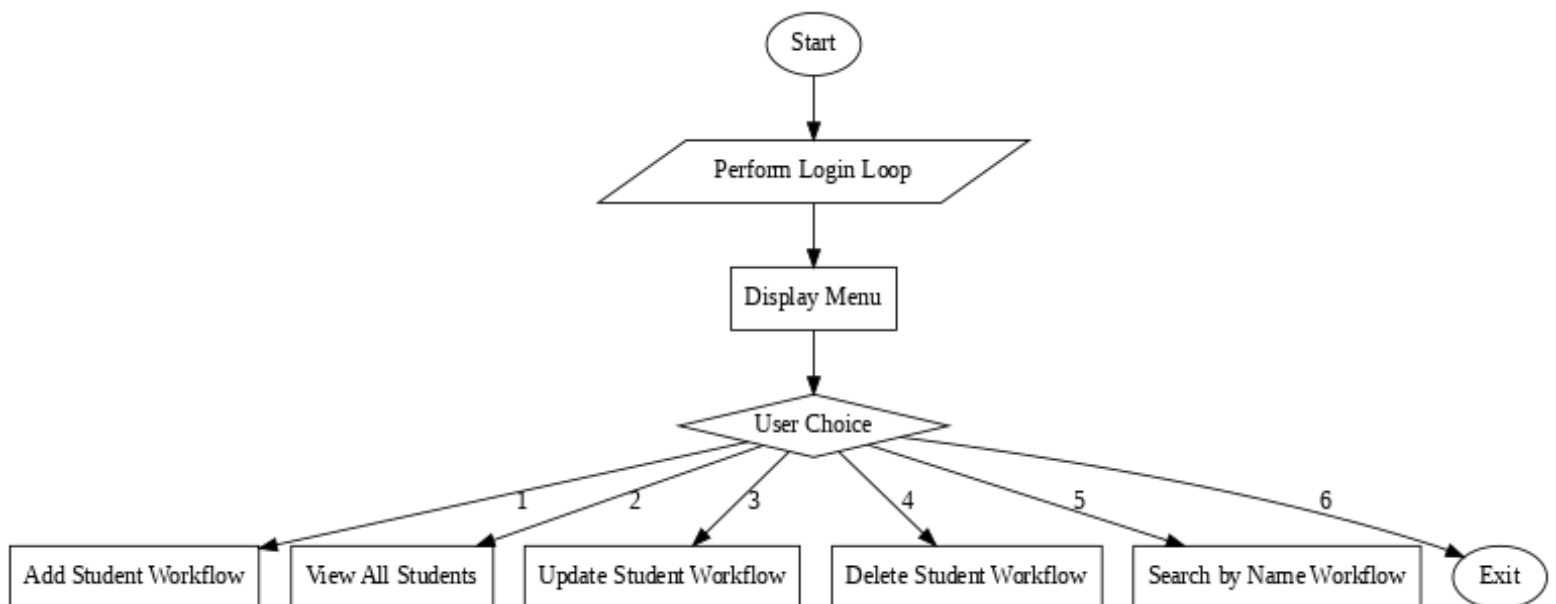
Delete Student: Removes a student from the system by their ID.

Search Student by Name: Finds and displays the details of a student based on their name.

Secure Login: Ensures only authorized users can access the system.

Requirement Analysis

Flowchart of Main Class:

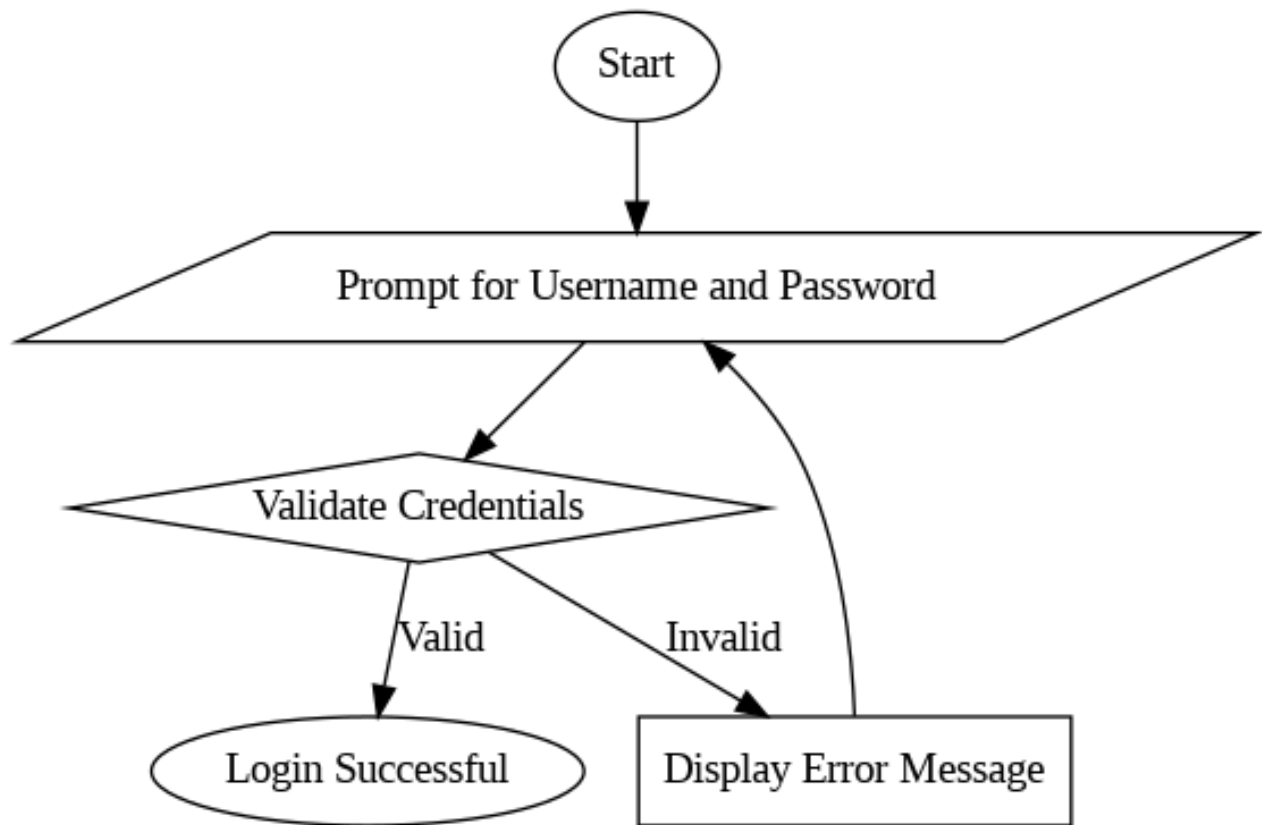


Algorithm of Main Class:

1. Start.
2. Perform the login operation through the *Login* class.

3. If login is successful, display the main menu with options to interact with the *StudentManagementSystem* class.
4. Based on user input, invoke methods from the *StudentManagementSystem* class to perform CRUD(create,read,update,delete) operations.
5. Repeat until the user chooses to exit.
6. End.

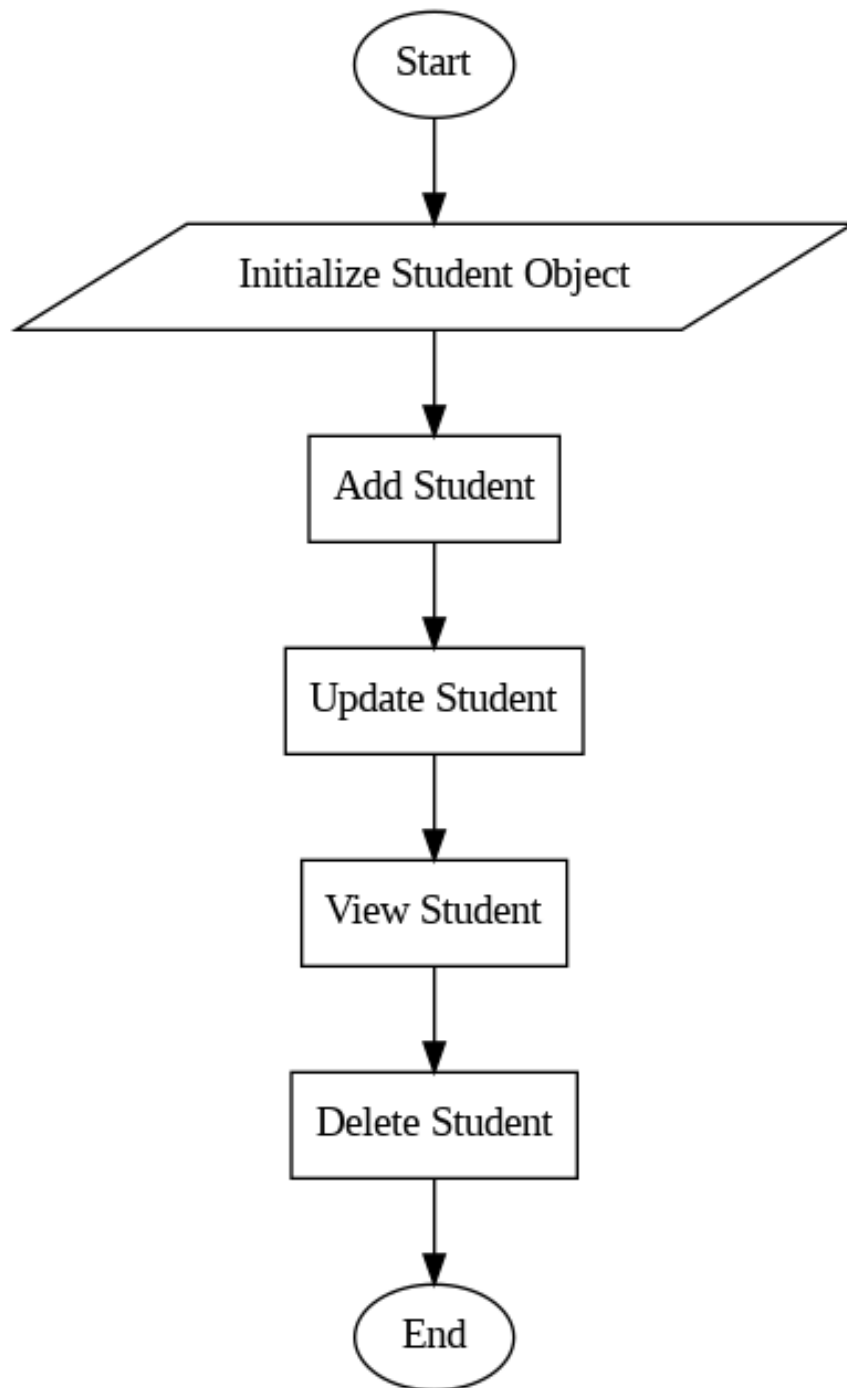
Flowchart of Login Class:



Algorithm of Login Class:

1. Start.
2. The user to enter the username and password.
3. Check if the entered username and password match the predefined credentials:
 - If correct: Display a success message and go to the main system.
 - If incorrect: Display an error message and go back to step 2.
4. End.

Flowchart of *Student* Class:

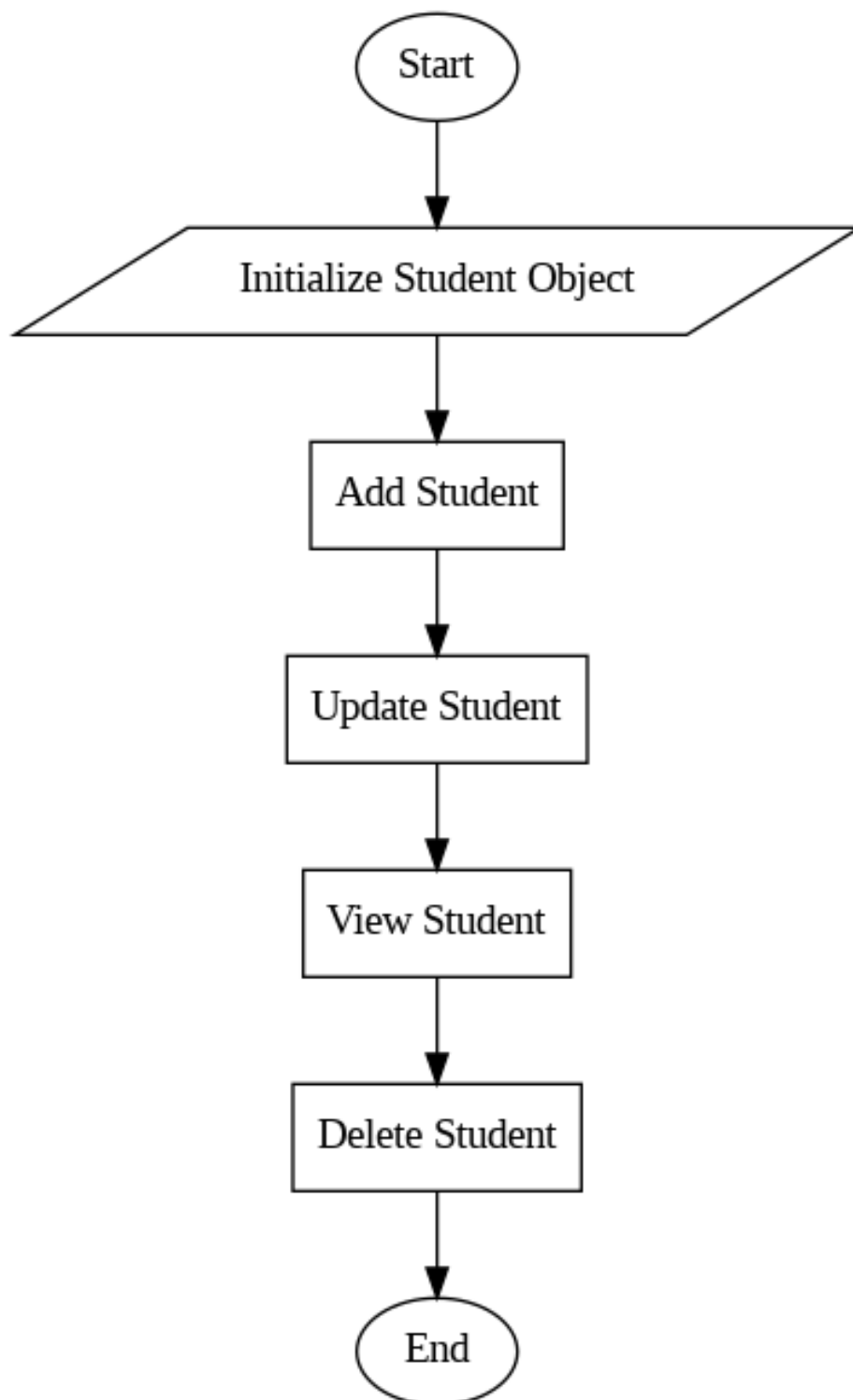


Algorithm of *Student* Class:

1. Start.
2. Add Student: Take student details (ID, Name, Contact Number, Course, Department). Store and display a success message.
3. View Student: Display stored details for the student.

4. Update Student: Modify student details (Name, Contact Number, Course, Department). Display the updated details.
5. Delete Student: Remove the student record and display a success message.
6. End.

Flowchart of *StudentManagementSystem* Class:



Algorithm of *Student Class*:

1. Start.
2. Display menu options:
 - Option 1: View all students.
 - Fetch all student records and display them in list format.
 - Go back to the menu.
 - Option 2: Search for a student by name.
 - Ask for the student name, search the list, and display the result.
 - If not found, display an error message.
 - Go back to the menu.
 - Option 3: Add a new student.
 - Take new student details, check for duplicate ID, and add them if valid.
 - Display the updated list.
 - Go back to the menu.
 - Option 4: Delete a student.
 - Ask for the student ID, search the list, and delete the record if found.
 - If not found, display an error message.
 - Go back to the menu.
 - Option 5: Exit the program.
 - Close the application.
3. End.

Classes:

- Main Class: The program's entry point, the main method, is located in the Main class. By creating an instance of the Login class, it causes the constructor of the Login class to start the login procedure. The user is presented with a menu to engage with the StudentManagementSystem class after successfully logging in.
- Login Class: By confirming the user's credentials, the Login class maintains security. Both the password and the username are predetermined. The application verifies the user's credentials after they are entered:

If valid: Creates an object of the StudentManagementSystem class and calls its methods.

If invalid: Prompts the user to re-enter their credentials until valid input is provided.

- *Student Class:* Individual student information, including ID, name, phone number, department, and course, is modeled by the Student class. It includes tools for examining, removing, updating, and adding student data. The system's student record management is modeled after this class.
- *StudentManagementSystem Class:* The StudentManagementSystem class handles the management of student records through CRUD operations. It maintains a dynamic list of student objects and provides the following functionalities:
 - Add a new student (with duplicate ID validation).
 - View all students.
 - Search for a student by name.
 - Update or delete student records.

Exception Class: There are two custom exception classes named DuplicateStudentException class and StudentNotFoundException Class. When a student with an existing ID is attempted to be added, a custom exception called the DuplicateStudentException class is triggered. It guarantees that each student's record in the system is unique. On the other hand, The StudentNotFoundException class is a custom exception that triggers when an operation (like update or delete) is performed on a student record that does not exist. It provides meaningful feedback to the user about missing records.

Technologies Used:

- Use of inheritance to make the program more readable and well maintainable.
- Inheritance consumes less memory.
- Inheritance makes the program more efficient and less time consuming.
- Use of equals() method to compare two strings which made less code to write.
- While loop helped to repeat the function class operation easily. It used Boolean value as parameter.
- Use of for loop to easily search index in static database to access information stored in that index.
- Use of switch case to choose between multiple options.

System Architecture:

The student management system is a terminal based application with a straightforward way to store and manage student data. The application is created using Java. The output and the application is operated through terminal.

Results:

Our program efficiently manages a large number of student records. We utilized inheritance and interfaces to enhance the program's maintainability and modularity. Core Java features such as the equals() method, switch-case statements, loops, dynamic arrays (ArrayList), and strings were implemented to provide robust functionality.

Each class in the program was equipped with constructors to establish a smooth workflow, enabling seamless interaction between different components. For managing the student database, we primarily relied on methods instead of constructors for better flexibility in handling operations.

The program is designed with security in mind, requiring a valid username and password for access. Its user-friendly interface ensures that administrators can perform tasks like adding, viewing, updating, deleting, and searching student records with ease. These operations are executed in minimal time, making the system efficient even when handling large datasets.

This system addresses the challenges educational institutions face in managing vast amounts of student information. By replacing conventional methods with this digital solution, the process becomes streamlined and less prone to errors. Additionally, the program's restricted access ensures that sensitive student data remains safe and secure.

This project demonstrates the practical application of programming principles and offers a reliable tool for student data management in real-world scenarios.

Output:

```
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\P
Login to Student Management System
Enter username: admin
Enter password: admin
Login successful!

Welcome to Student Management System:

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Search Student by Name
6. Exit
Enter your choice: |
```

```
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Search Student by Name
6. Exit
Enter your choice: 1
Enter Student ID: 1
Enter Name: Abid
Enter Contact Number: 8899000
Enter Department: ICE
Enter Course: ENG
Student added successfully!
```

```
Welcome to Student Management System:

1. Add Student|
2. View Students
3. Update Student
4. Delete Student
5. Search Student by Name
6. Exit
Enter your choice:
```

```
Student List:
Student ID: 99
Name: Mehedi Hasan
Contact Number: 187788282
Course: OOP
Department: ICE
-----
```

```
Student ID: 100
Name: Fahim Shahriar
Contact Number: 187785542
Course: OOP
Department: ICE|
-----
```

```
Student ID: 101
Name: Sadman Sakib
Contact Number: 187456282
Course: OOP
Department: ICE
-----
```

```
Student ID: 1
Name: Abid
Contact Number: 8899000
Course: ENG
Department: ICE
```

```
5. Search Student by Name
6. Exit
Enter your choice: 3
Enter Student ID to update: 99
Enter new Name: Mehedi Hassan
Enter new Contact Number: 5644446
Enter new Course: EEE
Student updated successfully!
```

Welcome to Student Management System:

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Search Student by Name
6. Exit

```
Enter your choice: 4
Enter Student ID to delete: 100
Student deleted successfully!
```

Welcome to Student Management System:

Conclusion:

Our program offers an efficient and secure system for managing student records, designed to simplify the administrative processes of educational institutions. Through the application of object-oriented programming concepts such as inheritance, interfaces, and exception handling, the program is modular, scalable, and maintainable. Key Java features like (`ArrayList`), switch-case statements, loops, and string operations enhance its functionality, making it a reliable and robust solution.

The program prioritizes security by implementing a login mechanism, ensuring that only authorized administrators can access and manage the data. Its user-friendly interface allows users to perform CRUD (Create, Read, Update, Delete) operations seamlessly, even with large datasets. The system effectively addresses the challenges associated with traditional methods of student information management, reducing errors and improving efficiency.

Future Work:

1. Database Integration: Replace in-memory storage with a database for persistent data management.
2. Role-Based Access: Add user roles (e.g., admin, teacher) with specific permissions.
3. Enhanced Security: Implement encryption and secure communication protocols.
4. Graphical Interface: Upgrade to a GUI for better usability.
5. Advanced Features: Include functionalities like attendance tracking, performance monitoring, and improved search/sort options.
6. Cloud Integration: Enable remote access and data sharing via cloud storage.